

Rapport DevOps — DevOps Django Bank

Module : DevOps

Projet : Application bancaire avec chaîne CI/CD

Étudiant : El-Mehdi Taoufik

Année universitaire : 2025-2026

Dépôt GitHub : <https://github.com/Hasanpiker5543/devops>

1. Introduction

Ce rapport présente la réalisation d'un mini-projet DevOps basé sur une application web de gestion bancaire. Le projet intègre Git, Docker, Docker Compose, tests automatisés et pipeline CI/CD GitHub Actions.

2. Présentation du sujet choisi

Le sujet choisi est une application bancaire permettant à un administrateur de gérer les clients et les comptes, et à un client de consulter ses comptes, effectuer des transferts et suivre ses transactions.

3. Fonctionnalités de l'application

L'application contient les fonctionnalités principales attendues pour une application bancaire moderne.

- Login
- Register
- Admin dashboard
- Client dashboard
- Gestion clients
- Gestion comptes
- Transferts
- Transactions
- Export PDF

4. Architecture de l'application

L'architecture suit le modèle Django MVT : Model, View, Template.

```
Utilisateur -> Navigateur -> Django Views -> Templates
Django Views -> Models -> SQLite
```

5. Choix techniques

Python, Django, SQLite, Docker, Docker Compose et GitHub Actions sont utilisés pour construire une application testable, reproductible et automatisée.

6. Modèle de données

Le modèle de données repose sur Client, Account et Transaction.

7. Mise en œuvre

Le dossier banking contient la logique métier et bank_project contient la configuration globale.

```
devops/
■■■ bank/
■■■ Dockerfile
■■■ docker-compose.yml
■■■ requirements.txt
■■■ .github/workflows/ci-cd.yml
```

8. Stratégie Git

Le projet est versionné avec Git et publié sur GitHub. La branche main contient la version stable.

9. Conteneurisation Docker

Un Dockerfile construit l'image de l'application et lance Django avec Gunicorn.

10. Docker Compose

docker-compose.yml expose le service web sur le port 8000 et utilise un volume pour SQLite.

```
docker compose up --build
```

11. Tests automatisés

Des tests Django vérifient les modèles et l'accès à la page de connexion.

```
cd bank
python manage.py test
```

12. Pipeline CI/CD

GitHub Actions installe les dépendances, exécute les checks, lance les tests, construit l'image Docker et publie l'image sur GHCR.

13. Difficultés rencontrées

La publication Docker a nécessité un tag en minuscules car Docker refuse les noms de dépôt avec majuscules.

14. Conclusion

Ce projet répond aux objectifs DevOps : application fonctionnelle, tests, Docker, Docker Compose, CI/CD et documentation.

15. Annexes

Dépôt GitHub : <https://github.com/Hasanpiker5543/devops>

Pipeline : <https://github.com/Hasanpiker5543/devops/actions/workflows/ci-cd.yml>